

Print Formatting

We can use the `print()` function to print out variables or strings:

In [1]:

```
print("hello")
```

```
[1] "hello"
```

In [2]:

```
x <- 10  
print(x)
```

```
[1] 10
```

In [3]:

```
x <- mtcars  
print(mtcars)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4
Merc 450SE	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3	3
Merc 450SL	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	3
Merc 450SLC	15.2	8	275.8	180	3.07	3.780	18.00	0	0	3	3
Cadillac Fleetwood	10.4	8	472.0	205	2.93	5.250	17.98	0	0	3	4
Lincoln Continental	10.4	8	460.0	215	3.00	5.424	17.82	0	0	3	4
Chrysler Imperial	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	4
Fiat 128	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1
Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	1
Toyota Corona	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	1
Dodge Challenger	15.5	8	318.0	150	2.76	3.520	16.87	0	0	3	2
AMC Javelin	15.2	8	304.0	150	3.15	3.435	17.30	0	0	3	2
Camaro Z28	13.3	8	350.0	245	3.73	3.840	15.41	0	0	3	4
Pontiac Firebird	19.2	8	400.0	175	3.08	3.845	17.05	0	0	3	2
Fiat X1-9	27.3	4	79.0	66	4.08	1.935	18.90	1	1	4	1
Porsche 914-2	26.0	4	120.3	91	4.43	2.140	16.70	0	1	5	2
Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.90	1	1	5	2
Ford Pantera L	15.8	8	351.0	264	4.22	3.170	14.50	0	1	5	4
Ferrari Dino	19.7	6	145.0	175	3.62	2.770	15.50	0	1	5	6
Maserati Bora	15.0	8	301.0	335	3.54	3.570	14.60	0	1	5	8
Volvo 142E	21.4	4	121.0	109	4.11	2.780	18.60	1	1	4	2

Formatting

We can format strings and variables together for printing in a few different ways:

`paste()`

The `paste()` function looks like this:

```
paste (... , sep = " ")
```

Where ... are the things you want to paste and **sep** is the separator you want between the pasted items, by default it is a space. For example:

In [12]:

```
print(paste('hello', 'world'))
```

```
[1] "hello world"
```

In [13]:

```
print(paste('hello', 'world', sep='-|-'))
```

```
[1] "hello-|-world"
```

paste0()

`paste0(..., collapse)` is equivalent to `paste(..., sep = "", collapse)`, slightly more efficiently.

In [10]:

```
paste0('hello', 'world')
```

Out[10]:

```
'helloworld'
```

sprintf

sprintf() is a wrapper for the C function `sprintf`, that returns a character vector containing a formatted combination of text and variable values. Meaning you can use % codes to place in variables by specifying all of them at the end. This is best shown through example:

In [15]:

```
sprintf("%s is %f feet tall\n", "Sven", 7.1)
```

Out[15]:

```
'Sven is 7.100000 feet tall '
```

In [16]:

```
# THIS WILL PRODUCE AN ERROR BECAUSE 7.1 is a float, not an integer  
sprintf("%s is %i feet tall\n", "Sven", 7.1)
```

Error in `sprintf("%s is %i feet tall\n", "Sven", 7.1)`: invalid format '`%i`'; use format `%f`, `%e`, `%g` or `%a` for numeric objects

So you are now wondering, what are the letters in front of % for? How do I know which ones to use? The full breakdown is available in the documentation:

In [17]:

```
help(sprintf)
```

Out[17]:

sprintf {base}	R Documentation
----------------	-----------------

Use C-style String Formatting Commands

Description

A wrapper for the C function `sprintf`, that returns a character vector containing a formatted combination of text and variable values.

Usage

```
sprintf(fmt, ...)
```

```
gettextf(fmt, ..., domain = NULL)
```

Arguments

<code>fmt</code>	a character vector of format strings, each of up to 8192 bytes.
<code>...</code>	values to be passed into <code>fmt</code> . Only logical, integer, real and character vectors are supported, but some coercion will be done: see the 'Details' section.
<code>domain</code>	see <code>gettext</code> .

Details

`sprintf` is a wrapper for the system `sprintf` C-library function. Attempts are made to check that the mode of the values passed match the format supplied, and R's special values (NA, Inf, -Inf and NaN) are handled correctly.

`gettextf` is a convenience function which provides C-style string formatting with possible translation of the format string.

The arguments (including `fmt`) are recycled if possible a whole number of times to the length of the longest, and then the formatting is done in parallel. Zero-length arguments are allowed and will give a zero-length result. All arguments are evaluated even if unused, and hence some types (e.g., "symbol" or "language", see `typeof`) are not allowed.

The following is abstracted from Kernighan and Ritchie (see References): however the actual implementation will follow the C99 standard and fine details (especially the behaviour under user error) may depend on the platform.

The string `fmt` contains normal characters, which are passed through to the output string, and also conversion specifications which operate on the arguments provided through `...`. The allowed conversion specifications start with a % and end with one of the letters in the set `aAdifeEgGosxX%`. These letters denote the following types:

d, i, o, x, X

Integer value, `o` being octal, `x` and `X` being hexadecimal (using the same case for `a-f` as the code). Numeric variables with exactly integer values will be coerced to integer. Formats `d` and `i` can also be used for logical variables, which will be converted to 0, 1 or NA.

f

Double precision value, in "fixed point" decimal notation of the form `"[-]mmm.ddd"`. The number of decimal places ("`d`") is specified by the precision: the default is 6; a precision of 0 suppresses the decimal point. Non-finite values are converted to NA, NaN or (perhaps a sign followed by) Inf.

e, E

Double precision value, in "exponential" decimal notation of the form `[-]m.ddde[+-]xx` or `[-]m.dddE[+-]xx`.

g, G

Double precision value, in `%e` or `%E` format if the exponent is less than -4 or greater than or equal to the precision, and `%f` format otherwise. (The precision (default 6) specifies the number of *significant* digits here, whereas in `%f`, `%e`, it is the number of digits after the decimal point.)

a, A

Double precision value, in binary notation of the form `[-]0xh.hhhp[+-]d`. This is a binary fraction expressed in hex multiplied by a (decimal) power of 2. The number of hex digits after the decimal point is specified by the precision: the default is enough digits to represent exactly the internal binary representation. Non-finite values are converted to NA, NaN or (perhaps a sign followed by) Inf. Format `%a` uses lower-case for `x`, `p` and the hex values: format `%A` uses upper-case.

This should be supported on all platforms as it is a feature of C99. The format is not uniquely defined: although it would be possible to make the leading `h` always zero or one, this is not always done. Most systems will suppress trailing zeros, but a few do not. On a

to make the leading 1 always zero or one, this is not always done. Most systems will suppress trailing zeros, but a few do not. On a well-written platform, for normal numbers there will be a leading one before the decimal point plus (by default) 13 hexadecimal digits, hence 53 bits. The treatment of denormalized (aka 'subnormal') numbers is very platform-dependent.

s

Character string. Character NAs are converted to "NA".

%

Literal % (none of the extra formatting characters given below are permitted in this case).

Conversion by `as.character` is used for non-character arguments with `s` and by `as.double` for non-double arguments with `f`, `e`, `E`, `g`, `G`. **NB:** the length is determined before conversion, so do not rely on the internal coercion if this would change the length. The coercion is done only once, so if `length(fmt) > 1` then all elements must expect the same types of arguments.

In addition, between the initial % and the terminating conversion character there may be, in any order:

m.n

Two numbers separated by a period, denoting the field width (`m`) and the precision (`n`).

-

Left adjustment of converted argument in its field.

+

Always print number with sign: by default only negative numbers are printed with a sign.

a space

Prefix a space if the first character is not a sign.

0

For numbers, pad to the field width with leading zeros. For characters, this zero-pads on some platforms and is ignored on others.

#

specifies "alternate output" for numbers, its action depending on the type: For `x` or `X`, `0x` or `0X` will be prefixed to a non-zero result. For `e`, `E`, `f`, `g` and `G`, the output will always have a decimal point; for `g` and `G`, trailing zeros will not be removed.

Further, immediately after % may come 1\$ to 99\$ to refer to numbered argument: this allows arguments to be referenced out of order and is mainly intended for translators of error messages. If this is done it is best if all formats are numbered: if not the unnumbered ones process the arguments in order. See the examples. This notation allows arguments to be used more than once, in which case they must be used as the same type (integer, double or character).

A field width or precision (but not both) may be indicated by an asterisk *: in this case an argument specifies the desired number. A negative field width is taken as a '-' flag followed by a positive field width. A negative precision is treated as if the precision were omitted. The argument should be integer, but a double argument will be coerced to integer.

There is a limit of 8192 bytes on elements of `fmt`, and on strings included from a single `%letter` conversion specification.

Field widths and precisions of `%s` conversions are interpreted as bytes, not characters, as described in the C standard.

The C doubles used for `R` numerical vectors have signed zeros, which `sprintf` may output as `-0`, `-0.000 ...`

Value

A character vector of length that of the longest input. If any element of `fmt` or any character argument is declared as UTF-8, the element of the result will be in UTF-8 and have the encoding declared as UTF-8. Otherwise it will be in the current locale's encoding.

Warning

The format string is passed down the OS's `sprintf` function, and incorrect formats can cause the latter to crash the `R` process. `R` does perform sanity checks on the format, but not all possible user errors on all platforms have been tested, and some might be terminal.

The behaviour on inputs not documented here is 'undefined', which means it is allowed to differ by platform.

Author(s)

Original code by Jonathan Rougier.

References

Kernighan, B. W. and Ritchie, D. M. (1988) *The C Programming Language*. Second edition, Prentice Hall. Describes the format options in table B-1 in the Appendix.

The C Standards, especially ISO/IEC 9899:1999 for 'C99'. Links can be found at <http://developer.r-project.org/Portability.html>.

`man sprintf on a Unix-alike system`

See Also

`formatC` for a way of formatting vectors of numbers in a similar fashion.

`paste` for another way of creating a vector combining text and values.

`gettext` for the mechanisms for the automated translation of text.

Examples

```
## be careful with the format: most things in R are floats
## only integer-valued reals get coerced to integer.

sprintf("%s is %f feet tall\n", "Sven", 7.1)      # OK
try(sprintf("%s is %i feet tall\n", "Sven", 7.1)) # not OK
  sprintf("%s is %i feet tall\n", "Sven", 7 )    # OK

## use a literal % :

sprintf("%.0f%% said yes (out of a sample of size %.0f)", 66.666, 3)

## various formats of pi :

sprintf("%f", pi)
sprintf("%.3f", pi)
sprintf("%1.0f", pi)
sprintf("%5.1f", pi)
sprintf("%05.1f", pi)
sprintf("%+f", pi)
sprintf("% f", pi)
sprintf("%-10f", pi) # left justified
sprintf("%e", pi)
sprintf("%E", pi)
sprintf("%g", pi)
sprintf("%g", 1e6 * pi) # -> exponential
sprintf("%.9g", 1e6 * pi) # -> "fixed"
sprintf("%G", 1e-6 * pi)

## no truncation:
sprintf("%1.f", 101)

## re-use one argument three times, show difference between %x and %X
xx <- sprintf("%1$d %1$x %1$X", 0:15)
xx <- matrix(xx, dimnames = list(rep("", 16), "%d%x%X"))
noquote(format(xx, justify = "right"))

## More sophisticated:

sprintf("min 10-char string '%10s'",
  c("a", "ABC", "and an even longer one"))

## Platform-dependent bad example from qdapTools 1.0.0:
## may pad with spaces or zeroes.
sprintf("%09s", month.name)

n <- 1:18
sprintf(paste0("e with %2d digits = %.", n, "g"), n, exp(1))

## Using arguments out of order
sprintf("second %2$1.0f, first %1$5.2f, third %3$1.0f", pi, 2, 3)

## Using asterisk for width or precision
sprintf("precision %.*f, width '%*.3f'", 3, pi, 8, pi)

## Asterisk and argument re-use, 'e' example reiterated:
```

```
sprintf("e with %1$2d digits = %2$.*1$g", n, exp(1))

## re-cycle arguments
sprintf("%s %d", "test", 1:3)

## binary output showing rounding/representation errors
x <- seq(0, 1.0, 0.1); y <- c(0,.1,.2,.3,.4,.5,.6,.7,.8,.9,1)
cbind(x, sprintf("%a", x), sprintf("%a", y))
```

[Package *base* version 3.2.2]

In []:
